

Simulação e Teste de Software (CC8550)

Aula 4: Introdução à Automação de Testes com pytest

Prof. Luciano Rossi

Ciência da Computação
Centro Universitário FEI

2º Semestre de 2026

Roteiro da Disciplina

- ❶ Fundamentos e Taxonomia de Testes de Software
- ❷ Ciclo de Vida de Teste de Software (STLC)
- ❸ Plano Mestre de Testes e Análise de Risco
- ❹ **Introdução à Automação de Testes com pytest**
- ❺ Teste de Unidade Avançado (FIRST, padrão AAA)
- ❻ Técnicas Caixa-Preta: Partição de Equivalência e Valor-Limite
- ❼ Técnicas Caixa-Preta: Teste Combinatorial e Property-Based
- ❽ Técnicas Caixa-Branca: Critérios de Cobertura Estrutural
- ❾ Testes de Mutação e Avaliação da Eficácia de Suítes de Teste
- ❿ Test-Driven Development (TDD)
- ⓫ Teste de Integração, Test Doubles e Níveis de Teste
- ⓬ Teste Orientado a Objetos e de Componentes

- 1 Motivação: por que automatizar testes
- 2 Instalação e convenções do pytest
- 3 Anatomia de um teste
- 4 Execução e interpretação de resultados
- 5 Testando exceções
- 6 Decoradores em Python
- 7 Fixtures: preparando o cenário
- 8 Parametrize: um teste, vários casos
- 9 Boas práticas

Motivação

Por que Automatizar Testes

Contexto

- Testar manualmente significa **repetir** os mesmos passos toda vez que o código muda: lento e sujeito a esquecimento;
- Automatizar significa escrever esses mesmos passos como código, que roda em **segundos** e sempre da mesma forma;
- **pytest** é a ferramenta padrão para automatizar testes em Python: vamos explorá-la a fundo hoje, do básico à injeção automática de dados via fixtures e parametrize.

Instalação e Convenções

O que é pytest

Definição

Definição

Framework para escrever e executar testes **automatizados** em Python: sintaxe simples baseada em assert, descoberta automática de arquivos de teste, mensagens de erro detalhadas quando um teste falha.

Instalação

```
pip install pytest
```

Convenções de nomenclatura

- **Arquivos** de teste: `test_*.py` ou `*_test.py`
- **Funções** de teste: `def test_*`
- **Classes** de teste: `class Test*`
- O pytest descobre esses arquivos e funções **automaticamente**, sem precisar registrá-los manualmente

Exemplo: Convenções

Exemplo

```
test_calculo.py  
calculo_test.py  
Testes_calculo.py  
calculo.py  
utils.py
```

Quais desses arquivos o pytest **descobre** como teste? Por que os outros ficam de fora?

Exemplo: Convenções

✓ Solução

Solução

- `test_calculo.py`: descoberto (`test_*.py`);
- `calculo_test.py`: descoberto (`*_test.py`);
- `Testes_calculo.py`: **não** descoberto (T maiúsculo não bate o padrão);
- `calculo.py` e `utils.py`: não descobertos, sem marcador de teste no nome.

Anatomia de um Teste

O que é assert

Definição

Definição

assert é a instrução que verifica se uma condição é **verdadeira**; se for falsa, Python levanta `AssertionError` e o pytest marca o teste como `FAILED`.

Variações de assert

Código

```
assert condicao
assert a == b
assert a in lista
assert isinstance(obj, Classe)
assert condicao, "mensagem customizada de erro"
```

Anatomia de um Teste

⌕ Código

```
# calculadora.py
def somar(a, b):
    return a + b
```

```
# test_calculadora.py
from calculadora import somar

def test_somar():
    assert somar(2, 3) == 5
```

Anatomia de um Teste

Conceito

- `import` **traz** a função a ser testada para o arquivo de teste;
- `def test_*`() nomeia o teste: o nome deve **descrever** o que está sendo verificado;
- `assert` compara o valor obtido com o valor esperado; se a expressão for falsa, o teste **falha**.

Exemplo: Anatomia de um Teste

Exemplo

```
def calcular_media(notas):  
    return sum(notas) / len(notas)  
  
def test_media():  
    notas = [7, 8, 9]  
    media = calcular_media(notas)  
    assert media == 8  
    assert media > 8.5
```

Este teste **passa** ou falha? Se falhar, em qual assert?

Exemplo: Anatomia de um Teste

✓ Solução

Solução

- A primeira asserção (`media == 8`) passa:
`sum([7,8,9])/3` é 8.0, e `8.0 == 8` é **verdadeiro**;
- A segunda (`media > 8.5`) falha: 8.0 não é maior que 8.5; o pytest **para** nesse assert e reporta FAILED.

Execução e Resultados

Executando o pytest

Código

```
pytest                                # roda tudo
pytest -v                             # modo verboso
pytest test_calc.py                   # so este arquivo
pytest test_calc.py::test_soma        # so esta funcao
```

Suíte de exemplo: test_calc.py (test_soma, test_subtrai) e test_string.py (test_maiuscula).

pytest x pytest -v

⌕ Código

```
$ pytest
test_calc.py ..
test_string.py .
3 passed in 0.04s
```

```
$ pytest -v
test_calc.py::test_soma PASSED
test_calc.py::test_subtrai PASSED
test_string.py::test_maiuscula PASSED
3 passed in 0.04s
```

Conceito

- Os **mesmos** 3 testes rodam nos dois casos: nenhum comando muda o que é executado, só o que é mostrado;
- Sem `-v`, cada arquivo aparece como uma linha de **pontos** (um ponto por teste que passou);
- Com `-v`, cada teste aparece **individualmente**, nomeado, o que ajuda a localizar qual falhou numa suíte grande.

Restringindo o Escopo

Código

```
$ pytest test_calc.py
test_calc.py ..
2 passed in 0.02s
```

```
$ pytest test_calc.py::test_soma
test_calc.py .
1 passed in 0.01s
```

Restringindo o Escopo

Conceito

- `test_calc.py` sozinho já **exclui** `test_string.py`: de 3 testes, só 2 rodam;
- `test_calc.py::test_soma` restringe ainda mais, a uma **única** função dentro do arquivo;
- Útil para rodar só o teste que está sendo escrito ou corrigido, sem **esperar** a suíte inteira.

Saída de uma Falha

Código

```
test_calc.py::test_soma FAILED  
  
AssertionError: assert 4 == 5  
1 failed in 0.03s
```


Interpretando o Resultado

Conceito

- PASSED: todas as asserções do teste foram **verdadeiras**;
- FAILED: pelo menos uma asserção foi **falsa** (o comportamento testado não bateu com o esperado);
- ERROR: uma exceção não prevista **interrompeu** o teste antes de chegar ao assert (ex.: erro ao preparar os dados);
- SKIPPED: o teste foi explicitamente **marcado** para não rodar nesta execução.

Exemplo: Execução

Exemplo

```
test_a.py: test_um (passa)
test_a.py: test_dois (assert falso)
test_b.py: test_tres (excecao antes do assert)
```

Rodando `pytest -v`, qual o resultado **reportado** para cada teste?

Exemplo: Execução

✓ Solução

Solução

- `test_um`: PASSED;
- `test_dois`: FAILED (a asserção não bateu);
- `test_tres`: ERROR (a exceção não prevista **interrompeu** o teste antes do assert).

Testando Exceções

Testando Exceções

⌕ Código

```
# calculo.py
def dividir(a, b):
    if b == 0:
        raise ValueError("divisao por zero")
    return a / b
```

Testando Exceções: os Testes

⌕ Código

```
# test_calculo.py
def test_dividir_por_zero():
    with pytest.raises(ValueError):
        dividir(10, 0)

def test_dividir_por_zero_mensagem():
    with pytest.raises(
        ValueError, match="divisao por zero"
    ):
        dividir(10, 0)
```

Como Funciona `pytest.raises()`

Conceito

- `pytest.raises(TipoDaExcecao)` é um gerenciador de contexto (`with`): ele **espera** que o código dentro do bloco levante aquela exceção;
- Se a exceção for levantada, o teste **passa**;
- Se o código terminar sem levantar nada, o teste **falha**, porque a exceção esperada não ocorreu;
- O parâmetro `match` compara o texto da exceção com uma expressão regular: só passa se a mensagem **também** bater, não só o tipo.

Exemplo: Exceções

⌕ Código

```
# conta.py
def sacar(saldo, valor):
    if valor > saldo:
        raise ValueError("saldo insuficiente")
    return saldo - valor
```


Exemplo: Exceções

Exemplo

```
# test_conta.py
def test_saque_a():
    with pytest.raises(ValueError):
        sacar(100, 50)

def test_saque_b():
    with pytest.raises(ValueError):
        sacar(100, 150)
```

Um dos dois testes **falha**. Qual, e por quê?

Exemplo: Exceções

✓ Solução

Solução

- `test_saque_a` **falha**: `sacar(100, 50)` não levanta exceção (50 não é maior que 100, saque válido); o `pytest.raises` nunca vê a exceção esperada;
- `test_saque_b` passa: `sacar(100, 150)` levanta `ValueError` corretamente, porque 150 é **maior** que o saldo.

Decoradores

O que é um Decorador

💡 Conceito

- Um decorador é uma função que recebe outra função e devolve uma versão dela com comportamento **extra**, sem alterar seu código original;
- A sintaxe `@nome_do_decorador` colocada antes de `def` **aplica** esse decorador à função logo abaixo;
- O `pytest` usa decoradores como `@pytest.fixture` e `@pytest.mark.parametrize` para dar significado **especial** a uma função comum.

Decorador na Prática

⌕ Código

```
def log_chamada(func):  
    def wrapper(*args):  
        print(f"chamando {func.__name__}")  
        return func(*args)  
    return wrapper  
  
@log_chamada  
def somar(a, b):  
    return a + b
```

O que a Sintaxe @ Faz

💡 Conceito

- `@log_chamada` acima de `def somar` é **equivalente** a escrever `somar = log_chamada(somar)` logo depois da função;
- Depois disso, chamar `somar(...)` na verdade executa `wrapper(...)`: o código extra roda, e só então a função original é chamada;
- `@pytest.fixture` e `@pytest.mark.parametrize` seguem o **mesmo** mecanismo: envolvem a função de teste com lógica adicional, sem que o autor do teste precise escrever isso à mão.

Exemplo: Decoradores

Exemplo

```
def dobrar_resultado(func):  
    def wrapper(*args):  
        return func(*args) * 2  
    return wrapper  
  
@dobrar_resultado  
def somar(a, b):  
    return a + b  
  
resultado = somar(3, 4)
```

Qual o **valor** final de resultado?

Exemplo: Decoradores

✓ Solução

Solução

- `somar(3, 4)` na verdade chama `wrapper(3, 4)`;
- Dentro do `wrapper`, `func(3, 4)` roda a soma
original: $3 + 4 = 7$;
- O `wrapper` multiplica esse resultado por 2 antes de devolver: `resultado = 14`.

Fixtures

Por que Fixtures

Contexto

- Vários testes costumam precisar do **mesmo** dado ou objeto inicial antes de rodar (ex.: uma lista de exemplo);
- Repetir esse preparo em cada função de teste é **redundante** e propenso a divergência entre os testes;
- Uma **fixture** é uma função que prepara esse cenário uma vez e é reaproveitada por quantos testes precisarem dela.

Sem Fixture: Repetição

⌕ Código

```
# test_lista.py (sem fixture)  
def test_soma():  
    lista = [1, 2, 3, 4, 5]  
    assert sum(lista) == 15  
  
def test_tamanho():  
    lista = [1, 2, 3, 4, 5]  
    assert len(lista) == 5
```

Com Fixture: a Fixture

⌕ Código

```
# conftest.py
import pytest

@pytest.fixture
def lista_exemplo():
    return [1, 2, 3, 4, 5]
```

Com Fixture: o Teste

⌕ Código

```
# test_lista.py
def test_soma(lista_exemplo):
    assert sum(lista_exemplo) == 15

def test_tamanho(lista_exemplo):
    assert len(lista_exemplo) == 5
```

Como o pytest Injeta a Fixture

Conceito

- O pytest **reconhece** `lista_exemplo` como fixture pelo decorador `@pytest.fixture`;
- Qualquer função de teste que declarar um parâmetro com o mesmo **nome** da fixture recebe o valor que ela retorna;
- A injeção é por nome, não por ordem: o pytest **procura** uma fixture com aquele nome exato.

Exemplo: Fixtures

⌕ Código

```
# conftest.py
import pytest

@pytest.fixture
def dados_exemplo():
    return [10, 20, 30]
```

Exemplo: Fixtures

Exemplo

```
# test_dados.py
def test_soma(dados_exemplos):
    assert sum(dados_exemplos) == 60
```

Este teste passa, falha ou dá **erro**? Por quê?

Exemplo: Fixtures

✓ Solução

Solução

- Dá ERROR: o parâmetro do teste se chama `dados_exemplos` (plural), mas a fixture definida é `dados_exemplo` (singular);
- O pytest não **encontra** nenhuma fixture com esse nome exato, e a injeção falha antes mesmo do teste rodar.

Parametrize

Por que Parametrize

Contexto

- Testar uma função com vários valores de entrada costuma significar **copiar** e colar o mesmo teste, trocando só o valor;
- Cada cópia aumenta a chance de **esquecer** de atualizar alguma delas quando o teste muda;
- `@pytest.mark.parametrize` roda a mesma função de teste uma vez para cada conjunto de valores, sem **duplicar** código.

A Função sob Teste

⌕ Código

```
# calculo.py
def quadrado(x):
    return x * x
```

Sem Parametrize: Repetição

❏ Código

```
# test_calculo.py (sem parametrize)  
def test_quadrado_positivo():  
    assert quadrado(2) == 4  
  
def test_quadrado_positivo_2():  
    assert quadrado(3) == 9  
  
def test_quadrado_negativo():  
    assert quadrado(-2) == 4
```

Com Parametrize: um Teste

⌕ Código

calculo.py não muda; só o teste fica mais **enxuto**.

```
# test_calculo.py
import pytest

@pytest.mark.parametrize("entrada, esperado", [
    (2, 4),
    (3, 9),
    (-2, 4),
])

def test_quadrado(entrada, esperado):
    assert quadrado(entrada) == esperado
```

Como o pytest Executa

💡 Conceito

- Cada tupla da lista vira uma execução **independente** de `test_quadrado`;
- No relatório, cada execução aparece **separada** (ex.: `test_quadrado[2-4]`), facilitando saber qual caso falhou;
- Esse mecanismo **volta** quando estudarmos como escolher bons valores de entrada (partição de equivalência e valor-limite), mais adiante no curso.

Exemplo: Parametrize

Exemplo

```
@pytest.mark.parametrize("entrada, esperado", [
    (2, 4),
    (5, 25),
    (4, 12),
])
def test_quadrado(entrada, esperado):
    assert quadrado(entrada) == esperado
```

Um dos três casos **falha**. Qual, e qual seria o erro?

Exemplo: Parametrize

✓ Solução

Solução

- O terceiro caso, $(4, 12)$, **falha**: `quadrado(4)` é 16, não 12;
- `AssertionError: assert 16 == 12`; os outros dois casos passam **normalmente**.

Boas Práticas

Checklist

- Testes independentes: cada um roda **sozinho**, em qualquer ordem
- Nomes descritivos: o nome do teste **explica** o que ele verifica
- Uma validação por teste: teste **focado** é mais fácil de diagnosticar
- Cubra casos **limites**: zero, negativos, valores máximos

Exemplo: Boas Práticas

Exemplo

```
contador = 0

def test_incrementa():
    global contador
    contador += 1
    assert contador == 1

def test_incrementa_de_novo():
    global contador
    contador += 1
    assert contador == 2
```

Qual boa prática da checklist este par de testes **viola**?

Exemplo: Boas Práticas

✓ Solução

Solução

- Viola **independência**: os dois testes compartilham o estado global contador;
- `test_incrementa_de_novo` só passa se `test_incrementa` rodar **antes** dele, nessa ordem; isolado ou em outra ordem, ele falha.

- pytest descobre testes **automaticamente** por convenção de nome, sem registro manual;
- Um teste resulta em PASSED, FAILED, ERROR ou SKIPPED, cada um com um significado **distinto**;
- `pytest.raises()` verifica que uma exceção esperada realmente **ocorre**;
- Fixtures **reaproveitam** cenários entre testes; `parametrize` roda o mesmo teste para vários casos sem duplicar código.

Simulação e Teste de Software (CC8550)

Aula 4: Introdução à Automação de Testes com pytest

Prof. Luciano Rossi

Ciência da Computação
Centro Universitário FEI

2º Semestre de 2026